PYTHON PROGRAMMING

ACTIVITY - BASED LEARNING

10 Case Studies with Solutions

Name	A Naganandini
Roll Number	26000854
Branch	Computer Science & Engineering (AI & ML)
Section	CSM1
Course	Python Programming
Institution	Dr. RVR NRI Institute of Technology

Submitted as part of the Python Programming activity-based learning work.

CONTENTS

Case Study 2: Progressive Electricity Billing

Case Study 3: Hospital Billing and Patient Classification

Case Study 4: Employee Payroll and Deduction Analysis

Case Study 5: Multi-Plan Telecommunications Cost Optimization

Case Study 8: Manufacturing Cost and Profitability Analysis

Case Study 9: Multi-Currency Travel Budget Analysis

Case Study 12: Attendance Recovery Computation

Case Study 14: Warehouse Inventory Reconciliation

Case Study 15: Smart Parking Fee Management

Case Study 16: Multi-Day Water Tank Simulation

CASE STUDY 2: Progressive Electricity Billing

1. Problem Statement

Calculate a household electricity bill using consumption slabs, a fixed charge, and a surcharge for high consumption.

2. Objectives

• Read the number of units consumed.

• Calculate the energy charge using the correct slab.

• Add fixed charges and any applicable surcharge.

• Display a clear bill summary.

3. Assumptions

• First 100 units cost Rs. 2 per unit.

• Next 100 units cost Rs. 3 per unit.

• Units above 200 cost Rs. 5 per unit.

• Fixed charge is Rs. 50.

• Consumption above 200 units attracts a 5% surcharge on the energy charge.

4. Algorithm

1. Start.

2. Input units consumed and validate that units are not negative.

3. Calculate energy charge according to the slab.

4. Calculate the fixed charge and surcharge.

5. Add the charges and display the bill.

6. Stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT units
IF units < 0:
    PRINT "Invalid units"
ELSE:
    IF units <= 100:
        energy = units * 2
    ELIF units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    ELSE:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5
    fixed = 50
    IF units > 200:
        surcharge = energy * 0.05
    ELSE:
        surcharge = 0
    total = energy + fixed + surcharge
    PRINT energy, fixed, surcharge, total
STOP
```

CASE STUDY 2: Progressive Electricity Billing (continued)

7. Python Source Code

```
units = int(input("Units consumed: "))

if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed = 50
    surcharge = energy * 0.05 if units > 200 else 0
    total = energy + fixed + surcharge

    print("Energy charge: Rs.", round(energy, 2))
    print("Fixed charge: Rs.", fixed)
    print("Surcharge: Rs.", round(surcharge, 2))
    print("Total bill: Rs.", round(total, 2))
```

8. Test Cases

Input / Situation	Expected Result	Purpose
80 units	Energy Rs. 160; total Rs. 210	Normal usage
150 units	Energy Rs. 350; total Rs. 400	Middle slab
250 units	Energy Rs. 750; surcharge Rs. 37.50; total Rs. 837.50	High usage

9. Conclusion

The program uses arithmetic and conditional statements to calculate a bill across progressive consumption slabs.

CASE STUDY 3: Hospital Billing and Patient Classification

1. Problem Statement

Calculate consultation, diagnostic, and treatment charges, apply a concession when eligible, validate the inputs, and classify the final bill.

2. Objectives

• Read patient age and the three charges.

• Reject negative values.

• Apply a concession for senior patients.

• Calculate and classify the final bill.

3. Assumptions

• All charges and age must be non-negative.

• Patients aged 60 or above receive a 10% concession on the subtotal.

• A final bill below Rs. 1,000 is Basic; Rs. 1,000 to below Rs. 5,000 is Standard; Rs. 5,000 or more is High.

4. Algorithm

1. Start.

2. Input age, consultation charge, diagnostic charge, and treatment charge.

3. Validate all values.

4. Add the charges to find the subtotal.

5. Apply a 10% concession if age is at least 60.

6. Find the final bill and its category.

7. Display the bill summary and stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT age, consultation, diagnostics, treatment
IF any input is negative:
    PRINT "Invalid input"
ELSE:
    subtotal = consultation + diagnostics + treatment
    IF age >= 60:
        concession = subtotal * 0.10
    ELSE:
        concession = 0
    final_bill = subtotal - concession
    IF final_bill < 1000:
        category = "Basic"
    ELIF final_bill < 5000:
        category = "Standard"
    ELSE:
        category = "High"
    PRINT subtotal, concession, final_bill, category
STOP
```

CASE STUDY 3: Hospital Billing and Patient Classification (continued)

7. Python Source Code

```
age = int(input("Patient age: "))
consultation = float(input("Consultation charge: Rs. "))
diagnostics = float(input("Diagnostic charge: Rs. "))
treatment = float(input("Treatment charge: Rs. "))

if age < 0 or consultation < 0 or diagnostics < 0 or treatment < 0:
    print("Invalid input")
else:
    subtotal = consultation + diagnostics + treatment
    concession = subtotal * 0.10 if age >= 60 else 0
    final_bill = subtotal - concession

    if final_bill < 1000:
        category = "Basic"
    elif final_bill < 5000:
        category = "Standard"
    else:
        category = "High"

    print("Subtotal: Rs.", round(subtotal, 2))
    print("Concession: Rs.", round(concession, 2))
    print("Final bill: Rs.", round(final_bill, 2))
    print("Category:", category)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Age 65; charges 500, 800, 2000	Subtotal 3300; concession 330; final Rs. 2970; Standard	Senior concession
Age 30; charges 200, 200, 300	Final Rs. 700; Basic	No concession
Age 50; consultation -5	Invalid input	Validation

9. Conclusion

The program demonstrates input validation, a conditional concession, and classification based on the final amount.

CASE STUDY 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives, then apply a deduction and prepare a salary statement.

2. Objectives

• Read basic pay, overtime hours, and incentive.

• Calculate allowances and overtime pay.

• Find gross salary, deduction, and net salary.

• Print a formatted salary statement.

3. Assumptions

• HRA is 20% of basic pay.

• Other allowance is 10% of basic pay.

• Overtime pay is Rs. 150 per hour.

• Incentive is added to gross salary.

• Deduction is 8% of gross salary.

4. Algorithm

1. Start.

2. Input basic pay, overtime hours, and incentive.

3. Validate that values are non-negative.

4. Calculate HRA, other allowance, and overtime pay.

5. Calculate gross salary, deduction, and net salary.

6. Display the salary statement and stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT basic, hours, incentive
IF basic < 0 OR hours < 0 OR incentive < 0:
    PRINT "Invalid input"
ELSE:
    hra = basic * 0.20
    other = basic * 0.10
    overtime = hours * 150
    gross = basic + hra + other + overtime + incentive
    deduction = gross * 0.08
    net = gross - deduction
    PRINT salary details
STOP
```

CASE STUDY 4: Employee Payroll and Deduction Analysis (continued)

7. Python Source Code

```
basic = float(input("Basic pay: Rs. "))
hours = float(input("Overtime hours: "))
incentive = float(input("Incentive: Rs. "))

if basic < 0 or hours < 0 or incentive < 0:
    print("Invalid input")
else:
    hra = basic * 0.20
    other = basic * 0.10
    overtime = hours * 150
    gross = basic + hra + other + overtime + incentive
    deduction = gross * 0.08
    net = gross - deduction

    print("--- Salary Statement ---")
    print("Basic pay: Rs.", round(basic, 2))
    print("HRA: Rs.", round(hra, 2))
    print("Other allowance: Rs.", round(other, 2))
    print("Overtime pay: Rs.", round(overtime, 2))
    print("Incentive: Rs.", round(incentive, 2))
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction: Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net, 2))
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Basic Rs. 20,000; 2 hours; incentive Rs. 500	Gross Rs. 26,800; deduction Rs. 2,144; net Rs. 24,656	Normal case
Basic Rs. 15,000; 0 hours; incentive 0	Gross Rs. 19,500; net Rs. 17,940	No overtime
Basic pay -100	Invalid input	Validation

9. Conclusion

The program calculates salary components and deductions using variables, arithmetic operators, and conditions.

CASE STUDY 5: Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Compare mobile plans against a user's monthly data usage and budget, calculate excess-data charges, and select the cheapest plan within budget.

2. Objectives

• Read monthly data usage and budget.

• Calculate the cost of each plan, including excess usage.

• Compare costs with the budget.

• Display the most economical eligible plan.

3. Assumptions

• Plan A costs Rs. 199 and includes 2 GB; extra data costs Rs. 50 per GB.

• Plan B costs Rs. 299 and includes 5 GB; extra data costs Rs. 30 per GB.

• Plan C costs Rs. 399 and includes 10 GB; extra data costs Rs. 20 per GB.

• Usage and budget cannot be negative.

4. Algorithm

1. Start.

2. Input data usage and budget.

3. Validate both values.

4. Calculate all three plan costs.

5. Find the lowest cost that does not exceed the budget.

6. Display the recommendation or state that no plan fits.

7. Stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT usage, budget
IF usage < 0 OR budget < 0:
    PRINT "Invalid input"
ELSE:
    cost_a = 199 + max(0, usage - 2) * 50
    cost_b = 299 + max(0, usage - 5) * 30
    cost_c = 399 + max(0, usage - 10) * 20
    plans = [("Plan A", cost_a), ("Plan B", cost_b), ("Plan C", cost_c)]
    best_name = ""
    FOR each name, cost in plans:
        IF cost <= budget AND (best_name is empty OR cost < best_cost):
            best_name = name
            best_cost = cost
    PRINT recommendation or no eligible plan
STOP
```

CASE STUDY 5: Multi-Plan Telecommunications Cost Optimization (continued)

7. Python Source Code

```
usage = float(input("Monthly data usage (GB): "))
budget = float(input("Monthly budget: Rs. "))

if usage < 0 or budget < 0:
    print("Invalid input")
else:
    cost_a = 199 + max(0, usage - 2) * 50
    cost_b = 299 + max(0, usage - 5) * 30
    cost_c = 399 + max(0, usage - 10) * 20
    plans = [("Plan A", cost_a), ("Plan B", cost_b), ("Plan C", cost_c)]

    best_name = ""
    best_cost = 0
    for name, cost in plans:
        if cost <= budget and (best_name == "" or cost < best_cost):
            best_name = name
            best_cost = cost

    for name, cost in plans:
        print(name, "cost: Rs.", round(cost, 2))
    if best_name == "":
        print("No plan is within your budget")
    else:
        print("Recommended plan:", best_name)
        print("Monthly cost: Rs.", round(best_cost, 2))
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Usage 4 GB; budget Rs. 350	Plan A costs Rs. 299; Plan B costs Rs. 299; cheapest eligible plan is A	Plan comparison
Usage 12 GB; budget Rs. 600	Plan C costs Rs. 439 and is eligible	High usage
Usage -1 GB	Invalid input	Validation

9. Conclusion

The program uses lists, loops, arithmetic, and conditions to compare mobile plans under a budget.

CASE STUDY 8: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate manufacturing costs from materials, labour, and overheads, then set a selling price that meets a target profit margin.

2. Objectives

• Read production costs and units produced.

• Calculate total cost and cost per unit.

• Add the target profit margin.

• Display the manufacturing summary.

3. Assumptions

• Material, labour, and overhead costs must be non-negative.

• Units produced must be greater than zero.

• The target profit is 20% of total manufacturing cost.

4. Algorithm

1. Start.

2. Input material cost, labour cost, overhead cost, and units produced.

3. Validate the inputs and prevent division by zero.

4. Calculate total cost and cost per unit.

5. Calculate target profit and selling price per unit.

6. Display the results and stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT material, labour, overhead, units
IF any cost < 0 OR units <= 0:
    PRINT "Invalid input"
ELSE:
    total_cost = material + labour + overhead
    cost_per_unit = total_cost / units
    profit = total_cost * 0.20
    total_revenue = total_cost + profit
    selling_price = total_revenue / units
    PRINT total_cost, cost_per_unit, profit, selling_price
STOP
```

CASE STUDY 8: Manufacturing Cost and Profitability Analysis (continued)

7. Python Source Code

```
material = float(input("Material cost: Rs. "))
labour = float(input("Labour cost: Rs. "))
overhead = float(input("Overhead cost: Rs. "))
units = int(input("Units produced: "))

if material < 0 or labour < 0 or overhead < 0 or units <= 0:
    print("Invalid input")
else:
    total_cost = material + labour + overhead
    cost_per_unit = total_cost / units
    profit = total_cost * 0.20
    total_revenue = total_cost + profit
    selling_price = total_revenue / units

    print("Total manufacturing cost: Rs.", round(total_cost, 2))
    print("Cost per unit: Rs.", round(cost_per_unit, 2))
    print("Target profit: Rs.", round(profit, 2))
    print("Selling price per unit: Rs.", round(selling_price, 2))
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Material 50000; labour 30000; overhead 20000; 1000 units	Total cost Rs. 100000; unit cost Rs. 100; selling price Rs. 120	Normal production
Units produced 0	Invalid input	Division-by-zero prevention
Material cost -10	Invalid input	Validation

9. Conclusion

The program calculates production costs, unit cost, and a selling price that includes the target profit.

CASE STUDY 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert an Indian Rupee travel budget into a selected foreign currency, include a conversion fee, compare planned expenses with the converted budget, and identify the largest expense category.

2. Objectives

• Read the INR budget, currency choice, fee percentage, and planned expenses.

• Convert the budget using the supplied exchange rate.

• Account for the conversion fee.

• Compare expenses with the available budget and find the largest category.

3. Assumptions

• Use the rates supplied for this exercise: 1 USD = Rs. 83, 1 EUR = Rs. 90, and 1 GBP = Rs. 105.

• Budget and expense values must be non-negative.

• Conversion fee is 2% of the converted budget.

• Expense amounts are entered in the selected foreign currency.

4. Algorithm

1. Start.

2. Input the INR budget, currency option, and expenses by category.

3. Validate the budget, option, and expense values.

4. Convert the INR budget using the selected rate.

5. Subtract the 2% conversion fee to find the usable budget.

6. Compare total planned expenses with the usable budget.

7. Find and display the largest expense category and budget status.

8. Stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT budget_inr, choice, expenses_by_category
VALIDATE all inputs
IF choice == 1: rate = 83; currency = "USD"
ELIF choice == 2: rate = 90; currency = "EUR"
ELIF choice == 3: rate = 105; currency = "GBP"
ELSE: PRINT "Invalid currency choice"
converted = budget_inr / rate
fee = converted * 0.02
usable_budget = converted - fee
total_expense = sum(expenses)
largest_category = category with maximum expense
IF total_expense <= usable_budget:
    status = "Within budget"
ELSE:
    status = "Budget exceeded"
PRINT currency, usable_budget, total_expense, largest_category, status
STOP
```

CASE STUDY 9: Multi-Currency Travel Budget Analysis (continued)

7. Python Source Code

```
budget_inr = float(input("Travel budget in INR: "))
print("Currency: 1. USD  2. EUR  3. GBP")
choice = int(input("Choose currency: "))

categories = ["Stay", "Food", "Transport", "Shopping"]
expenses = []
for category in categories:
    amount = float(input(category + " expense in selected currency: "))
    expenses.append(amount)

if budget_inr < 0 or any(amount < 0 for amount in expenses) or choice not in [1, 2, 3]:
    print("Invalid input")
else:
    if choice == 1:
        currency, rate = "USD", 83
    elif choice == 2:
        currency, rate = "EUR", 90
    else:
        currency, rate = "GBP", 105

    converted = budget_inr / rate
    fee = converted * 0.02
    usable_budget = converted - fee
    total_expense = sum(expenses)
    largest_index = expenses.index(max(expenses))
    largest_category = categories[largest_index]

    print("Budget after conversion fee:", round(usable_budget, 2), currency)
    print("Total planned expenses:", round(total_expense, 2), currency)
    print("Largest expense category:", largest_category)
    if total_expense <= usable_budget:
        print("Status: Within budget")
    else:
        print("Status: Budget exceeded")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Budget Rs. 83,000; USD; expenses 300, 200, 150, 100	Converted budget 1000 USD; fee 20 USD; usable 980 USD; total 750 USD; within budget	Normal conversion
Budget Rs. 45,000; EUR; expenses 200, 150, 100, 100	Converted budget 500 EUR; usable 490 EUR; total 550 EUR; budget exceeded	Budget check
Negative food expense	Invalid input	Validation

9. Conclusion

The program demonstrates currency conversion, a fee calculation, list processing, and budget comparisons. Exchange rates are fixed classroom assumptions, not live rates.

CASE STUDY 12: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentage and determine how many consecutive classes a student must attend to reach a required attendance threshold.

2. Objectives

• Read classes held, classes attended, and the required percentage.

• Calculate current attendance.

• Check whether the student already meets the threshold.

• Calculate the minimum consecutive classes needed if attendance is below the threshold.

3. Assumptions

• Total classes held must be greater than zero.

• Classes attended must be between zero and total classes held.

• The eligibility threshold is supplied as a percentage from 0 to 100.

• Future classes are attended without any absences.

4. Algorithm

1. Start.

2. Input total classes, attended classes, and threshold.

3. Validate all values.

4. Calculate current attendance percentage.

5. If the threshold is met, report zero additional classes.

6. Otherwise increase the future attended and total counts until the threshold is met.

7. Display the current percentage and required classes, then stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT total, attended, threshold
IF total <= 0 OR attended < 0 OR attended > total OR threshold < 0 OR threshold > 100:
    PRINT "Invalid input"
ELSE:
    current = attended / total * 100
    extra = 0
    WHILE (attended + extra) / (total + extra) * 100 < threshold:
        extra = extra + 1
    PRINT current, extra
STOP
```

CASE STUDY 12: Attendance Recovery Computation (continued)

7. Python Source Code

```
total = int(input("Total classes held: "))
attended = int(input("Classes attended: "))
threshold = float(input("Required attendance percentage: "))

if (total <= 0 or attended < 0 or attended > total
        or threshold < 0 or threshold > 100):
    print("Invalid input")
else:
    current = attended / total * 100
    extra = 0
    while (attended + extra) / (total + extra) * 100 < threshold:
        extra += 1

    print("Current attendance:", round(current, 2), "%")
    print("Consecutive classes needed:", extra)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Total 40; attended 28; threshold 75%	Current 70%; 8 consecutive classes needed	Recovery calculation
Total 50; attended 40; threshold 75%	Current 80%; 0 classes needed	Already eligible
Attended 45; total 40	Invalid input	Validation

9. Conclusion

The program uses percentages and a while loop to calculate the minimum future classes needed to meet an attendance requirement.

CASE STUDY 14: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, incoming quantities, dispatched quantities, and recorded closing stock for several products, detect differences, and calculate replenishment needs.

2. Objectives

• Read inventory data for each product.

• Calculate expected closing stock.

• Compare expected and recorded closing stock.

• Report discrepancies and the quantity to reorder.

3. Assumptions

• All quantities must be non-negative integers.

• Expected closing stock equals opening stock plus incoming stock minus dispatched stock.

• A discrepancy exists when expected and recorded closing stock differ.

• The minimum desired stock level is 20 units for each product.

4. Algorithm

1. Start.

2. Input product details and stock quantities.

3. Validate that quantities are non-negative and dispatch does not exceed available stock.

4. Calculate expected closing stock.

5. Compare it with recorded closing stock.

6. Calculate replenishment needed to reach 20 units.

7. Display each product's reconciliation result and stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT number_of_products
FOR each product:
    INPUT name, opening, incoming, dispatched, recorded_closing
    IF any quantity < 0 OR dispatched > opening + incoming:
        PRINT "Invalid inventory data"
    ELSE:
        expected = opening + incoming - dispatched
        difference = recorded_closing - expected
        reorder = max(0, 20 - expected)
        IF difference == 0:
            status = "Matched"
        ELSE:
            status = "Discrepancy found"
        PRINT name, expected, difference, reorder, status
STOP
```

CASE STUDY 14: Warehouse Inventory Reconciliation (continued)

7. Python Source Code

```
products = [
    ("Notebook", 50, 20, 30, 40),
    ("Pen", 100, 50, 80, 65),
    ("Folder", 25, 10, 20, 15)
]

minimum_stock = 20
for name, opening, incoming, dispatched, recorded in products:
    if min(opening, incoming, dispatched, recorded) < 0:
        print(name, "Invalid quantity")
    elif dispatched > opening + incoming:
        print(name, "Invalid dispatch quantity")
    else:
        expected = opening + incoming - dispatched
        difference = recorded - expected
        reorder = max(0, minimum_stock - expected)

        print("\nProduct:", name)
        print("Expected closing stock:", expected)
        print("Recorded closing stock:", recorded)
        if difference == 0:
            print("Status: Matched")
        else:
            print("Discrepancy:", difference, "units")
        print("Replenishment needed:", reorder)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Opening 50; incoming 20; dispatched 30; recorded 40	Expected 40; matched; reorder 0	Matched stock
Opening 100; incoming 50; dispatched 80; recorded 65	Expected 70; discrepancy -5; reorder 0	Stock discrepancy
Opening 10; incoming 0; dispatched 5; recorded 5	Expected 5; reorder 15	Replenishment

9. Conclusion

The program checks inventory arithmetic, identifies differences between calculated and recorded stock, and flags replenishment needs.

CASE STUDY 15: Smart Parking Fee Management

1. Problem Statement

Calculate parking charges using entry and exit times, vehicle category, overnight parking, daily limits, and penalties while checking time inputs.

2. Objectives

• Read vehicle type and entry and exit times.

• Validate the time values and parking duration.

• Calculate the fee according to vehicle category and duration.

• Apply daily limits and any penalty, then display the total.

3. Assumptions

• Times are entered in 24-hour HH:MM format on the same day; an exit earlier than entry is treated as invalid.

• Car rate is Rs. 30 per hour and bike rate is Rs. 15 per hour.

• Any part of an hour is charged as a full hour.

• Maximum regular charge is Rs. 200 per day.

• A fixed Rs. 100 penalty applies if the vehicle stays more than 12 hours.

4. Algorithm

1. Start.

2. Input vehicle type, entry time, and exit time.

3. Validate the type and time values.

4. Convert both times into minutes and calculate duration.

5. Round duration up to whole hours.

6. Calculate the vehicle charge and apply the daily cap.

7. Add the long-stay penalty if applicable.

8. Display duration and final fee, then stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT vehicle_type, entry_time, exit_time
VALIDATE HH:MM times and vehicle type
entry_minutes = convert entry time to minutes
exit_minutes = convert exit time to minutes
IF exit_minutes < entry_minutes:
    PRINT "Invalid time range"
ELSE:
    duration = exit_minutes - entry_minutes
    hours = ceiling(duration / 60)
    IF vehicle_type == "car": rate = 30
    ELSE: rate = 15
    regular_fee = min(hours * rate, 200)
    penalty = 100 IF duration > 720 ELSE 0
    total = regular_fee + penalty
    PRINT duration, total
STOP
```

CASE STUDY 15: Smart Parking Fee Management (continued)

7. Python Source Code

```
from math import ceil

vehicle = input("Vehicle type (car/bike): ").lower()
entry = input("Entry time (HH:MM): ")
exit_time = input("Exit time (HH:MM): ")

def to_minutes(value):
    parts = value.split(":")
    if len(parts) != 2:
        return -1
    hour, minute = int(parts[0]), int(parts[1])
    if hour < 0 or hour > 23 or minute < 0 or minute > 59:
        return -1
    return hour * 60 + minute

start = to_minutes(entry)
end = to_minutes(exit_time)

if vehicle not in ["car", "bike"] or start < 0 or end < 0 or end < start:
    print("Invalid input")
else:
    duration = end - start
    hours = ceil(duration / 60)
    rate = 30 if vehicle == "car" else 15
    regular_fee = min(hours * rate, 200)
    penalty = 100 if duration > 12 * 60 else 0
    total = regular_fee + penalty

    print("Parking duration:", duration // 60, "hours", duration % 60, "minutes")
    print("Regular fee: Rs.", regular_fee)
    print("Penalty: Rs.", penalty)
    print("Total parking fee: Rs.", total)
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Car; 09:00 to 11:10	3 billable hours; fee Rs. 90	Hourly charge
Bike; 08:00 to 22:30	15 billable hours; capped fee Rs. 200 plus Rs. 100 penalty	Long stay
Car; 18:00 to 17:00	Invalid time range	Validation

9. Conclusion

The program uses string handling, time conversion, conditions, and arithmetic to calculate parking fees and penalties.

CASE STUDY 16: Multi-Day Water Tank Simulation

1. Problem Statement

Simulate water levels over several days by updating tank storage from changing inflow and outflow quantities, while preventing the stored amount from going below zero or above capacity.

2. Objectives

• Read tank capacity and starting water level.

• Process daily inflow and outflow values.

• Update the water level after each day.

• Report overflow, shortage, and the final water level.

3. Assumptions

• Tank capacity and initial water level must be non-negative.

• Initial water level cannot exceed capacity.

• Daily inflow and outflow values must be non-negative.

• Water above capacity is reported as overflow and is not stored.

• If outflow exceeds available water plus inflow, the unmet amount is reported as shortage.

4. Algorithm

1. Start.

2. Input tank capacity, initial level, and number of days.

3. Validate the initial values.

4. For each day, input inflow and outflow.

5. Calculate available water and identify any shortage or overflow.

6. Update the tank level within the allowed range.

7. Display the daily and final results, then stop.

5. Flowchart (Plain Text)

```
START -> INPUT DETAILS -> VALIDATE INPUTS -> CALCULATE / APPLY CONDITIONS -> DISPLAY RESULT -> STOP
```

6. Pseudocode

```
START
INPUT capacity, level, days
VALIDATE capacity, level, days
FOR day from 1 to days:
    INPUT inflow, outflow
    available = level + inflow
    overflow = max(0, available - capacity)
    available = min(available, capacity)
    shortage = max(0, outflow - available)
    level = max(0, available - outflow)
    PRINT day, level, overflow, shortage
PRINT final level
STOP
```

CASE STUDY 16: Multi-Day Water Tank Simulation (continued)

7. Python Source Code

```
capacity = float(input("Tank capacity (litres): "))
level = float(input("Initial water level (litres): "))
days = int(input("Number of days: "))

if capacity < 0 or level < 0 or level > capacity or days < 1:
    print("Invalid input")
else:
    for day in range(1, days + 1):
        inflow = float(input("Day " + str(day) + " inflow: "))
        outflow = float(input("Day " + str(day) + " outflow: "))

        if inflow < 0 or outflow < 0:
            print("Invalid inflow or outflow")
            break

        available = level + inflow
        overflow = max(0, available - capacity)
        available = min(available, capacity)
        shortage = max(0, outflow - available)
        level = max(0, available - outflow)

        print("Day", day, "water level:", round(level, 2), "litres")
        print("Overflow:", round(overflow, 2), "litres")
        print("Shortage:", round(shortage, 2), "litres")

    print("Final water level:", round(level, 2), "litres")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Capacity 1000; initial 500; inflow 300; outflow 200	Final level 600 litres; no overflow or shortage	Normal update
Capacity 1000; initial 900; inflow 250; outflow 100	Overflow 150 litres; final level 900 litres	Overflow check
Capacity 500; initial 100; inflow 0; outflow 150	Shortage 50 litres; final level 0 litres	Shortage check

9. Conclusion

The program models tank levels over multiple days and uses conditions and loops to track overflow, shortage, and remaining water.